QA Basics Notes and Test Scenarios

Introduction to QA

Quality Assurance (QA) is the process of checking whether a software application works correctly and meets the expected requirements before it is released to users.It is a systematic approach designed to ensure that products and services consistently meet customer expectations, achieve service targets, and adhere to industry standards. QA helps improve software quality, reduce bugs, and provide a better user experience.

Goals of QA

- Find bugs before users find them.
- Improve software quality.
- Ensure application stability and security.
- Verify requirements are working correctly.
- Increase customer satisfaction.

SDLC (Software Development Life Cycle)

The Software Development Life Cycle (SDLC) is a well-structured process that guides software development projects from start to finish. It provides a clear framework for planning, building, and maintaining software, ensuring that development is systematic and meets quality standards.SDLC, is the process of planning, writing, modifying, and maintaining software.SDLC is the process used to develop software step by step.

Phases of SDLC

1. Requirement Gathering
2. Planning
3. Design
4. Development
5. Testing
6. Deployment

7. Maintenance

STLC (Software Testing Life Cycle)

The Software Testing Life Cycle (STLC) is a systematic sequence of phases executed by Quality Assurance (QA) teams to validate software quality. STLC is the testing process followed by QA teams.It ensures the application meets requirements and detects defects early, consisting of Requirement Analysis, Test Planning, Test Design, Environment Setup, Execution, etc.

Phases of STLC

1. Requirement Analysis
2. Test Planning
3. Test Case Creation
4. Test Environment Setup
5. Test Execution
6. Bug Reporting
7. Test Closure

Types of Testing

1. Functional Testing

Checks whether the application features work according to requirements.

Example:

- Login button works correctly
- User can register successfully

2. Smoke Testing

Basic testing done to check whether the application is stable for further testing.

Example:

- App opens successfully
- Main pages load correctly

3. Sanity Testing

Checks whether a specific feature or bug fix works correctly after small changes.

Example:

- Verify password reset works after bug fix

4. Regression Testing

Ensures old features still work after adding new updates.

Example:

- Login still works after payment feature update

5. Negative Testing

Checks how the application behaves with invalid input.

Example:

- Enter wrong password during login
- Submit empty registration form

6. UI Testing

Checks user interface elements like buttons, fonts, colors, and alignment.

7. Compatibility Testing

Checks whether the application works on different devices, browsers, and operating systems.

Bug Life Cycle

1. New
2. Assigned
3. Open
4. Fixed
5. Retest
6. Close

Difference Between Severity and Priority

Severity	Priority
How serious the bug is	How quickly the bug should be fixed
Related to functionality impact	Related to business urgency
Set by tester	Usually set by manager/team lead

Example:

- App crash during payment = High Severity + High Priority
- Typo in About Us page = Low Severity + Low Priority

Components of a Test Case

Field	Description
Test Case ID	Unique identifier
Test Scenario	What needs to be test

Preconditions	Conditions before testing
Test Steps	Actions performed
Test Data	Input values used
Expected Result	Expected output
Actual Result	Actual application behavior
Status	Pass or Fail

Ensures old features still work after adding new updates.

Example:

- Login still works after payment feature update

5. Negative Testing

Checks how the application behaves with invalid input.

Example:

- Enter wrong password during login
- Submit empty registration form

6. UI Testing

Checks user interface elements like buttons, fonts, colors, and alignment.

7. Compatibility Testing

Checks whether the application works on different devices, browsers, and operating systems.